TECNOLÓLICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE ENERO – AGOSTO 2026

CARRERA

Ing. Sistemas computacionales

MATERIA

Patrones de diseño

TÍTULO

Actividad de evidencia 2_U2 Singleton

UNIDAD A EVALUAR

Unidad 2

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO

Castillejo Robles Lennyn Alejandro 22210880

NOMBRE DEL MAESTRO

Maribel Guerrero Luis

FECHA DE ENTREGA

09 de marzo del 2026

Contenido

Instrucción 3
Atributo estático de la instancia 3
¿Qué significa cada palabra? 3
¿Para qué sirve? 3
Constructor 3
¿Qué significa cada parte? 3
¿Para qué sirve hacerlo privado? 3
Diagramas UML 4
¿Por qué no se pueden crear múltiples objetos? 5
Código 6
Conclusión 10

Instrucción

El Singleton es un patrón de diseño que tiene un único propósito: asegurarse de que una clase solo pueda crear un objeto durante toda la vida del programa. Esto es útil cuando necesitas un recurso compartido, como una central de emergencias, una conexión a base de datos o una configuración global.

Atributo estático de la instancia

private static Central_911 _instance;

¿Qué significa cada palabra?

- **private** → Nadie de afuera puede tocarlo directamente, solo la propia clase lo controla
- **static** → Pertenece a la clase en general, no a un objeto en específico. Existe aunque no se haya creado ningún objeto todavía
- **Central**_911 → Es el tipo de dato, o sea, guarda una referencia a un objeto de tipo Central_911
- **_instance** → Es la variable que guarda la única copia de la central

¿Para qué sirve?

Es como una caja fuerte donde se guarda la única central que existe. Antes de crear una nueva central, el programa revisa si esa caja ya tiene algo adentro. Si ya tiene algo, usa esa misma, si está vacía, crea una nueva y la guarda ahí.

Constructor

```
private Central_911()
{
    Central = "Central 911";
}
```

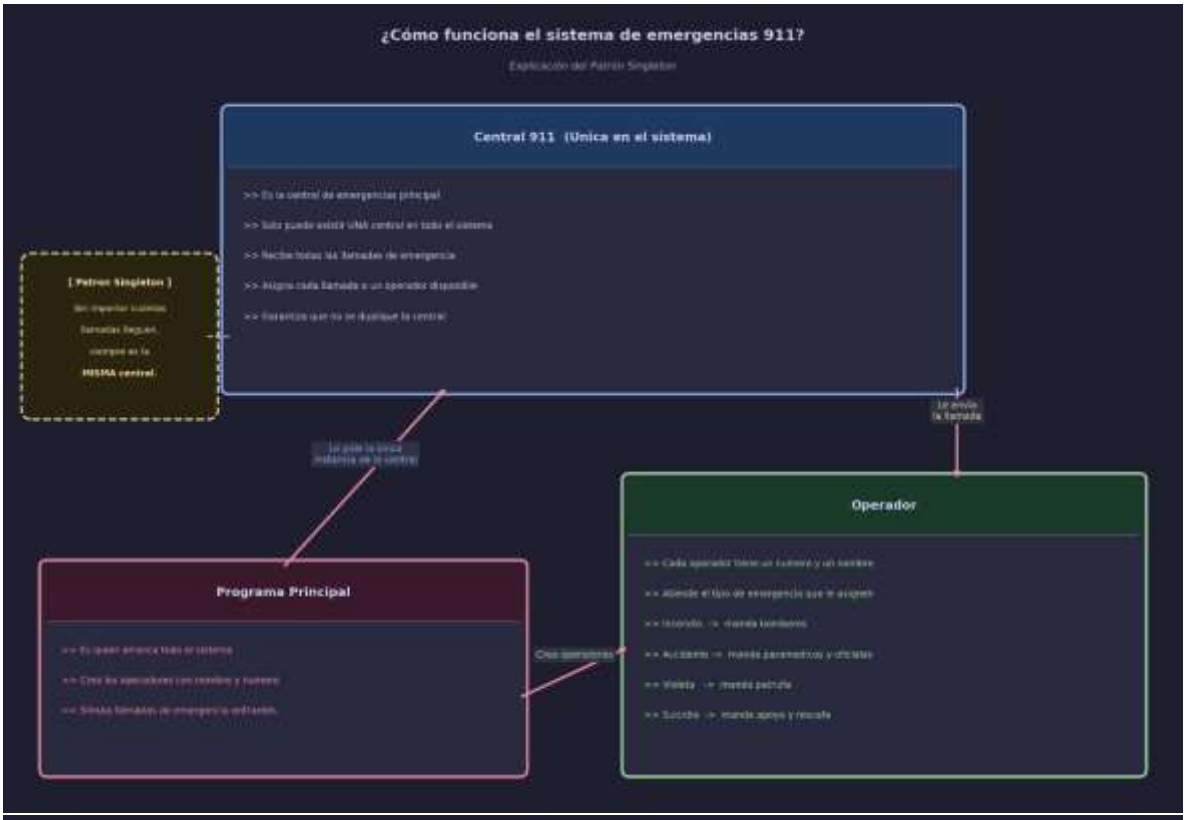
¿Qué significa cada parte?

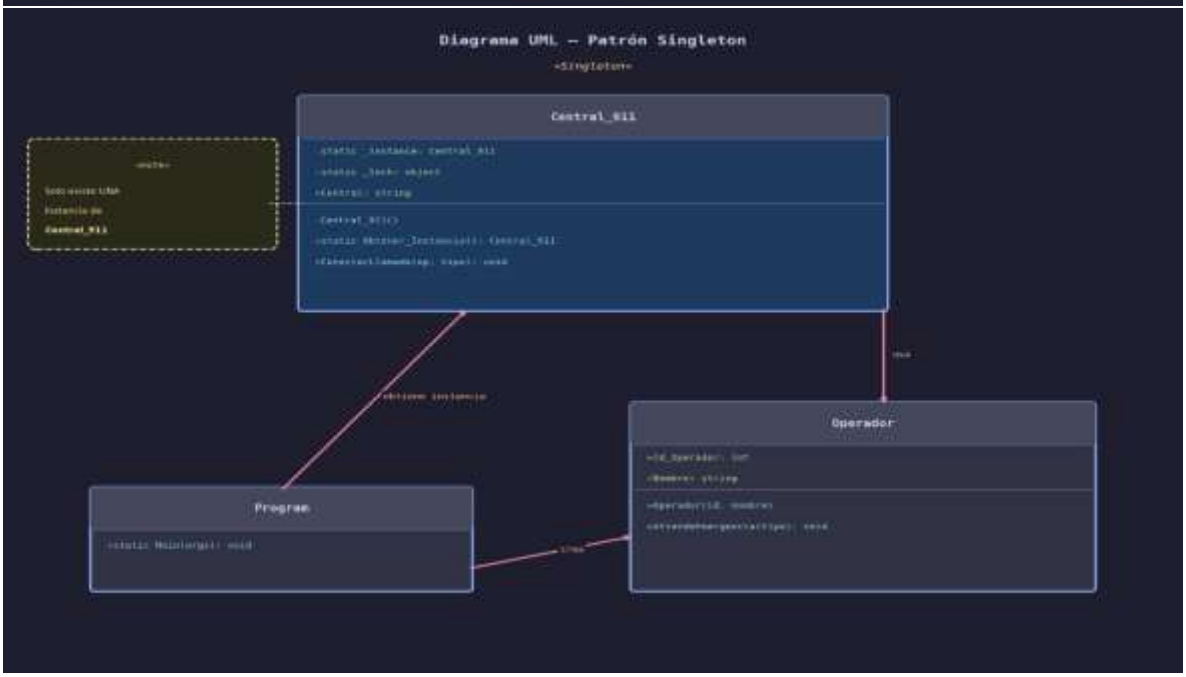
- **private** → Nadie de afuera puede crear una Central_911 directamente. Si alguien escribe new Central_911() desde otro lado, el programa lanza un error
- **Central_911**() → Es el constructor, es decir, las instrucciones que se ejecutan cuando se crea el objeto
- **Central = "Central 911"** → Le asigna el nombre a la central cuando se crea

¿Para qué sirve hacerlo privado?

Es la clave del Singleton. Al hacer el constructor privado, bloqueas que alguien pueda crear más de una central accidentalmente. La única forma de obtener la central es a través del método Obtener_Instancia(), que es quien controla que solo exista una.

Diagramas UML





¿Por qué no se pueden crear múltiples objetos?

Porque el código tiene dos mecanismos que trabajan juntos para impedirlo:

El constructor es privado: private Central_911(), cuando es constructor es private nadie de a fuera puede escribir esto: Central_911 miCentral = new Central_911(); // ERROR de compilación

El programa directamente no deja compilar si se intenta crear un objeto así.

También el atributo estatico hace de guardia private static Central_911 _instance;, combinado con este método:

```
if (_instance == null)

{

    _instance = new Central_911(); // Solo se ejecuta UNA vez

}

return _instance; // Las demas veces devuelve la misma
```

La primera vez que alguien pide la central, _instance está vacío entonces la crea y la guarda. La segunda, tercera y todas las veces siguientes, _instance ya tiene algo entonces devuelve la misma que ya existe, nunca crea una nueva.

Código

```
public class Central_911
{
    private static Central_911 _instance;
    private static readonly object _lock = new object();

    public string Central { get; private set; }

    private Central_911()
    {
        Central = "Central 911";
    }

    public static Central_911 Obtener_Instancia()
    {
        if (_instance == null)
        {
            lock (_lock)
            {
                if (_instance == null)
                {
                    _instance = new Central_911();
                }
            }
        }
        return _instance;
    }

    public void ConectarLlamada(Operador operador, string tipoEmergencia)
    {
        Console.WriteLine("\nLlamada conectada con el operador " + operador.Nombre);
        operador.AtiendeEmergencia(tipoEmergencia);
    }
}
```

```
1  public class Operador
2  {
3      public int Id_Operador { get; set; }
4      public string Nombre { get; set; }
5
6      public Operador(int id, string nombre)
7      {
8          Id_Operador = id;
9          Nombre = nombre;
10     }
11
12     public void AtiendeEmergencia(string tipoEmergencia)
13     {
14         Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo: {tipoEmergencia}");
15
16         switch (tipoEmergencia)
17         {
18             case "Intento de suicidio":
19                 Console.WriteLine("Enviando unidades de apoyo y rescate");
20                 break;
21             case "Incendio":
22                 Console.WriteLine("Enviando bomberos.");
23                 break;
24             case "Accidente":
25                 Console.WriteLine("Enviado paramedicos y oficiales.");
26                 break;
27             case "Violeta":
28                 Console.WriteLine("Enviando una patrulla.");
29                 break;
30             case "Robo en progreso":
31                 Console.WriteLine("Enviando unidades de  patrulla.");
32                 break;
33             case "Accidente de tráfico":
34                 Console.WriteLine("Enviando unidades de ambulancia.");
35                 break;
36             default:
37                 Console.WriteLine("Tipo de emergencia no reconocido.");
38                 break;
39         }
40     }
41 }
```

```
1  public class Operador
2  {
3      public int Id_Operador { get; set; }
4      public string Nombre { get; set; }
5
6      public Operador(int id, string nombre)
7      {
8          Id_Operador = id;
9          Nombre = nombre;
10     }
11
12     public void AtiendeEmergencia(string tipoEmergencia)
13     {
14         Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo: {tipoEmergencia}");
15
16         switch (tipoEmergencia)
17         {
18             case "Intento de suicidio":
19                 Console.WriteLine("Enviando unidades de apoyo y rescate");
20                 break;
21             case "Incendio":
22                 Console.WriteLine("Enviando bomberos.");
23                 break;
24             case "Accidente":
25                 Console.WriteLine("Enviado paramedicos y oficiales.");
26                 break;
27             case "Violeta":
28                 Console.WriteLine("Enviando una patrulla.");
29                 break;
30             case "Robo en progreso":
31                 Console.WriteLine("Enviando unidades de  patrulla.");
32                 break;
33             case "Accidente de tráfico":
34                 Console.WriteLine("Enviando unidades de ambulancia.");
35                 break;
36             default:
37                 Console.WriteLine("Tipo de emergencia no reconocido.");
38                 break;
39         }
40     }
41 }
```

```
Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador María
Operador María atendiendo emergencia de tipo: Accidente
Enviado paramedicos y oficiales.

Llamada conectada con el operador Pedro
Operador Pedro atendiendo emergencia de tipo: Accidente de tráfico
Enviando unidades de ambulancia.

Llamada conectada con el operador Jorge
Operador Jorge atendiendo emergencia de tipo: Robo en progreso
Enviando unidades de  patrulla.

Llamada conectada con el operador Jorge
Operador Jorge atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate
Operador Jorge atendiendo emergencia de tipo: Intento de suicidio
Operador Jorge atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate
```

Conclusión

Con esta práctica entendí cómo funciona el patrón Singleton y para qué sirve. Lo que más me llamó la atención fue que con solo dos cosas, el constructor privado y el atributo estático, puedes controlar completamente cuántos objetos se crean de una clase. En el caso de la Central 911 tiene mucho sentido porque no tendría lógica que existieran dos centrales al mismo tiempo, todas las llamadas tienen que llegar al mismo lugar. Creo que este patrón es útil en situaciones donde necesitas que un recurso sea compartido por todo el programa sin duplicarse.